EXPERIMENT - 71

1. Problem Statement

Write a C program to dynamically allocate two matrices and perform addition, subtraction, and multiplication using pointer notation. Implement each operation as a separate function and release all allocated memory properly.

2. Algorithm

Main Program

Step 1: START
Step 2: Read r1, c1, allocate matrix A with allocate_matrix() and read its elements. Do the same for matrix B with r2 and c2.
Step 3: If r1 = r2 and c1 = c2, call add_matrices() and subtract_matrices(), display each result and free it. Otherwise display that the sizes must be the same.
Step 4: If c1 = r2, call multiply_matrices(), display the result and free it. Otherwise display that the columns of A must equal the rows of B.
Step 5: Free A and B with free_matrix().
Step 6: STOP

Functions

allocate_matrix(rows, cols): Allocate an array of rows pointers, then allocate cols integers for each row. Return it.
add_matrices() and subtract_matrices(): Allocate the result. For every i and j, set *(*(result + i) + j) to the sum or the difference of *(*(a + i) + j) and *(*(b + i) + j).
multiply_matrices(): Allocate the result of r1 × c2. For every i and j, set the element to 0 and add *(*(a + i) + k) * *(*(b + k) + j) for k = 0 to c1 − 1.
free_matrix(m, rows): Free each row, then free the array of pointers.

3. Test Case Table

#	Matrix A	Matrix B	Expected Output	Actual Output	Result
1	2×2: 1 2 3 4	2×2: 5 6 7 8	Sum of A and B: 6 8 / 10 12 Difference of A and B: -4 -4 / -4 -4 Product of A and B: 19 22 / 43 50	Sum of A and B: 6 8 / 10 12 Difference of A and B: -4 -4 / -4 -4 Product of A and B: 19 22 / 43 50	PASS
2	2×3: 1 2 3 4 5 6	3×2: 7 8 9 10 11 12	Addition and subtraction need matrices of the same size. Product of A and B: 58 64 / 139 154	Addition and subtraction need matrices of the same size. Product of A and B: 58 64 / 139 154	PASS
3	2×2: 2 3 4 5	2×3: 1 2 3 4 5 6	Addition and subtraction need matrices of the same size. Product of A and B: 14 19 24 / 24 33 42	Addition and subtraction need matrices of the same size. Product of A and B: 14 19 24 / 24 33 42	PASS
4	2×3: 1 2 3 4 5 6	2×3: 6 5 4 3 2 1	Sum of A and B: 7 7 7 / 7 7 7 Difference of A and B: -5 -3 -1 / 1 3 5 / Multiplication needs the columns of A to equal the rows of B.	Sum of A and B: 7 7 7 / 7 7 7 Difference of A and B: -5 -3 -1 / 1 3 5 / Multiplication needs the columns of A to equal the rows of B.	PASS

4. Source Code

matrix71.c

```
/* Experiment 71: two dynamically allocated matrices with addition,
   subtraction and multiplication, all using pointer notation. */
#include <stdio.h>
#include <stdlib.h>

int **allocate_matrix(int rows, int cols);
void read_matrix(int **m, int rows, int cols);
void display_matrix(int **m, int rows, int cols);
int **add_matrices(int **a, int **b, int rows, int cols);
int **subtract_matrices(int **a, int **b, int rows, int cols);
int **multiply_matrices(int **a, int **b, int r1, int c1, int c2);
void free_matrix(int **m, int rows);

int main(void)
{
    int r1, c1, r2, c2;
    int **a, **b, **result;

    printf("Enter the rows and columns of matrix A: ");
    scanf("%d %d", &r1, &c1);
    a = allocate_matrix(r1, c1);
    printf("Enter the elements of matrix A: ");
    read_matrix(a, r1, c1);

    printf("Enter the rows and columns of matrix B: ");
    scanf("%d %d", &r2, &c2);
    b = allocate_matrix(r2, c2);
    printf("Enter the elements of matrix B: ");
    read_matrix(b, r2, c2);

    if (r1 == r2 && c1 == c2) {
        result = add_matrices(a, b, r1, c1);
        printf("Sum of A and B:\n");
        display_matrix(result, r1, c1);
        free_matrix(result, r1);

        result = subtract_matrices(a, b, r1, c1);
        printf("Difference of A and B:\n");
        display_matrix(result, r1, c1);
        free_matrix(result, r1);
    } else
        printf("Addition and subtraction need matrices of the same size.\n");

    if (c1 == r2) {
        result = multiply_matrices(a, b, r1, c1, c2);
        printf("Product of A and B:\n");
        display_matrix(result, r1, c2);
        free_matrix(result, r1);
    } else
        printf("Multiplication needs the columns of A to equal the rows of B.\n");

    free_matrix(a, r1);
    free_matrix(b, r2);
    return 0;
}
```

```
int **allocate_matrix(int rows, int cols)
{
    int i;
    int **m = (int **)malloc(rows * sizeof(int *));

    if (m == NULL) {
        printf("Memory allocation failed.\n");
        exit(1);
    }
    for (i = 0; i < rows; i++) {
        *(m + i) = (int *)malloc(cols * sizeof(int));
        if (*(m + i) == NULL) {
            printf("Memory allocation failed.\n");
            exit(1);
        }
    }
    return m;
}

void read_matrix(int **m, int rows, int cols)
{
    int i, j;
    for (i = 0; i < rows; i++)
        for (j = 0; j < cols; j++)
            scanf("%d", *(m + i) + j);
}

void display_matrix(int **m, int rows, int cols)
{
    int i, j;
    for (i = 0; i < rows; i++)
        for (j = 0; j < cols; j++)
            printf("%d%c", *(*(m + i) + j), j == cols - 1 ? '\n' : ' ');
}

int **add_matrices(int **a, int **b, int rows, int cols)
{
    int i, j;
    int **sum = allocate_matrix(rows, cols);
    for (i = 0; i < rows; i++)
        for (j = 0; j < cols; j++)
            *(*(sum + i) + j) = *(*(a + i) + j) + *(*(b + i) + j);
    return sum;
}

int **subtract_matrices(int **a, int **b, int rows, int cols)
{
    int i, j;
    int **diff = allocate_matrix(rows, cols);
    for (i = 0; i < rows; i++)
        for (j = 0; j < cols; j++)
            *(*(diff + i) + j) = *(*(a + i) + j) - *(*(b + i) + j);
    return diff;
}

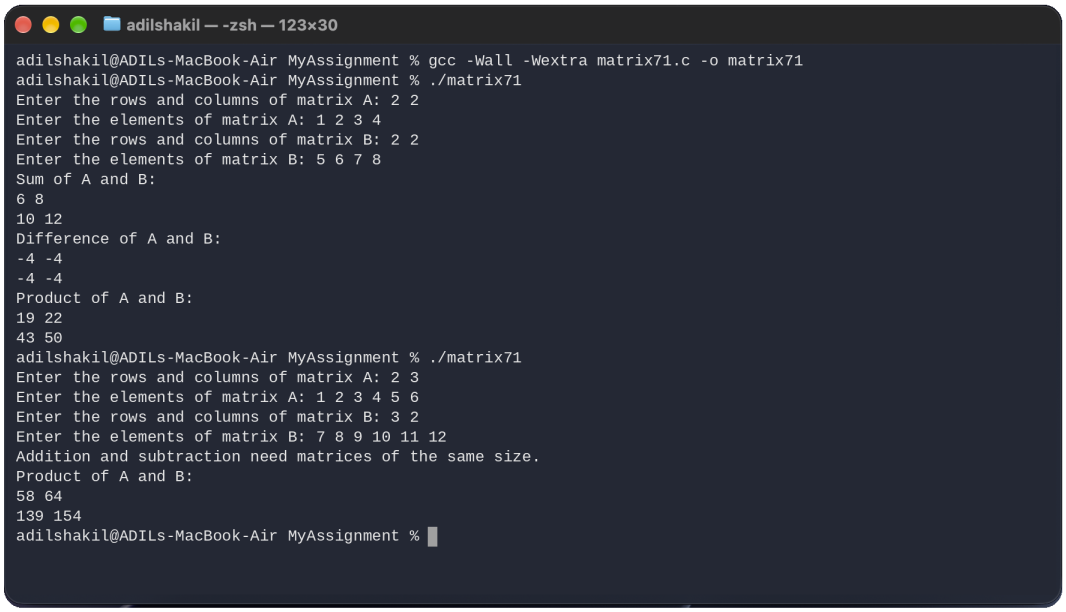
int **multiply_matrices(int **a, int **b, int r1, int c1, int c2)
{
    int i, j, k;
    int **product = allocate_matrix(r1, c2);
    for (i = 0; i < r1; i++)
        for (j = 0; j < c2; j++) {
            *(*(product + i) + j) = 0;
            for (k = 0; k < c1; k++)
                *(*(product + i) + j) += *(*(a + i) + k) * *(*(b + k) + j);
        }
    return product;
}

void free_matrix(int **m, int rows)
{
    int i;
    for (i = 0; i < rows; i++)
        free(*(m + i));
    free(m);
}
```

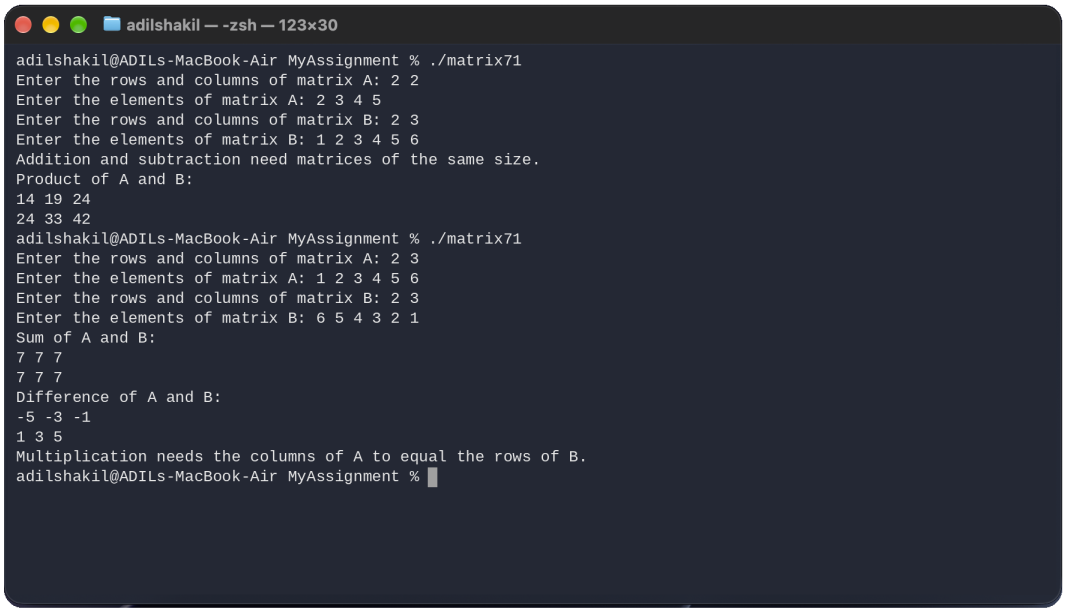
5. Compilation and Output

Compiled with `gcc -Wall -Wextra matrix71.c -o matrix71` so the small warnings would show up too. It built clean.

Cases 1 and 2



Cases 3 and 4



All four passed.